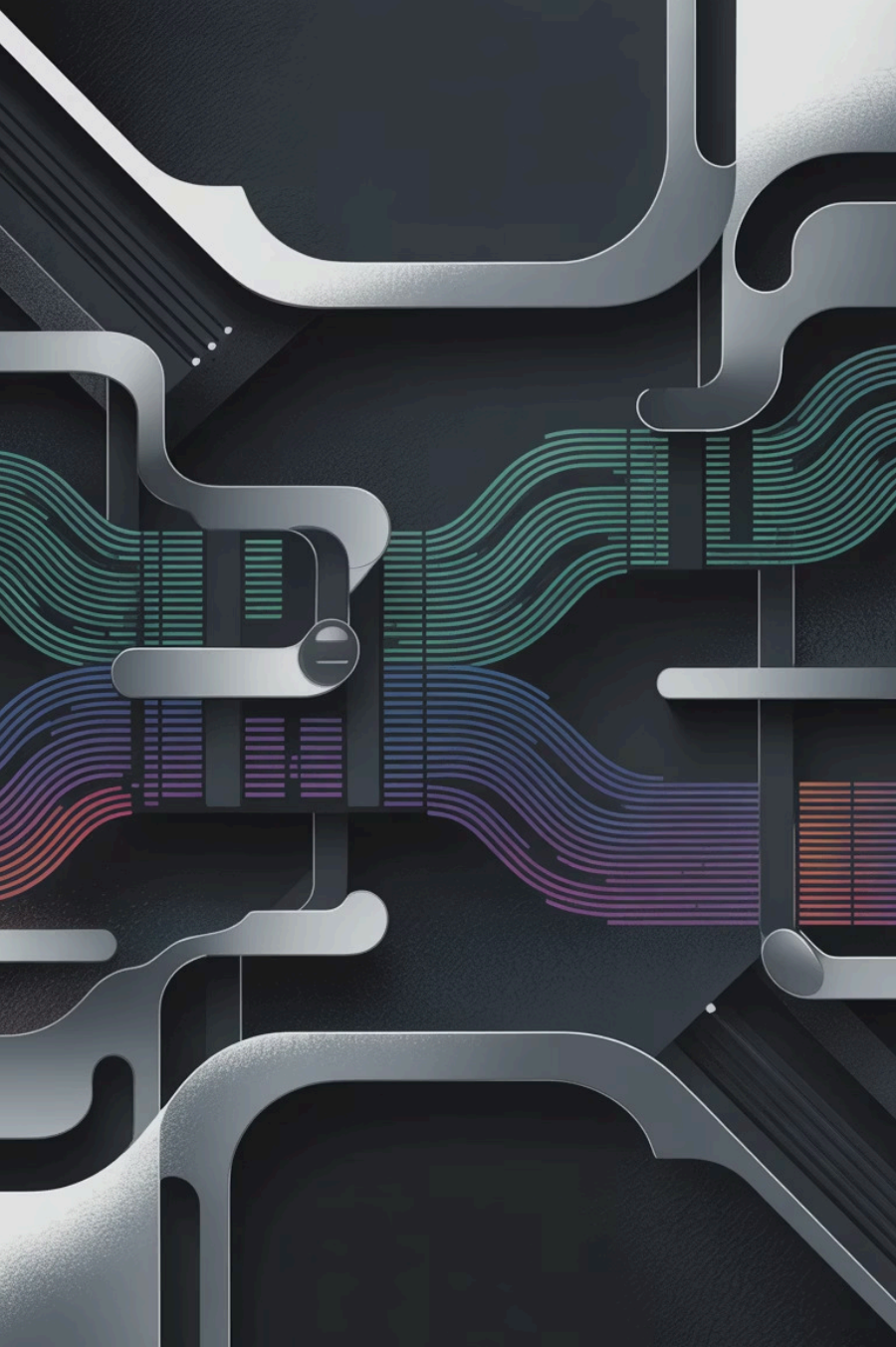




Module 02 - Data Engineering for LLM Pipelines

Building the Foundation for AI Engineering Excellence

Key Concepts

01

Python + NumPy Fundamentals

Refresh array operations, vectorization, and memory optimization techniques essential for processing large datasets

02

Jupyter for Data Exploration

Master notebook workflows, visualization magic commands, and best practices for reproducible analysis

03

Hands-On Lab Practice

Apply concepts through practical exercises building data pipelines for LLM preprocessing

Provides the foundational toolkit to extract, transform, and prepare data for LLM fine-tuning workflows with confidence and efficiency.

Why NumPy Matters for LLM Pipelines

Performance at Scale

NumPy's C-based implementation makes it 10-100x faster than pure Python loops when processing millions of tokens or embeddings. This speed advantage becomes critical when preparing training datasets with billions of parameters.

Vectorized operations process entire arrays in single instructions, eliminating the overhead of Python's interpreter loop. For LLM data pipelines handling gigabytes of text, this efficiency difference is transformative.

Memory Efficiency

NumPy arrays consume significantly less memory than Python lists—a crucial advantage when working with large language model datasets. A float64 array uses 8 bytes per element versus 24+ bytes for Python list objects.

This compact representation enables processing larger batches in memory, reducing disk I/O and accelerating the entire pipeline. Understanding memory footprint helps you optimize for both speed and resource constraints.

NumPy Array Creation Essentials

1

From Python Lists

```
import numpy as np
arr = np.array([1, 2, 3, 4, 5])
matrix = np.array([[1, 2], [3, 4]])
```

The most straightforward method for small datasets or when converting existing Python structures.

2

Initialized Arrays

```
zeros = np.zeros((3, 4)) # 3x4 array of zeros
ones = np.ones((2, 3))  # 2x3 array of ones
empty = np.empty((3, 2)) # Uninitialized values
```

Use these for pre-allocating memory when you'll fill values later—much faster than appending.

3

Range and Random

```
range_arr = np.arange(0, 10, 2) # [0, 2, 4, 6, 8]
rng = np.random.default_rng(42)
rng.random((3, 3)) # was np.random.rand(3, 3)
rng.standard_normal(1000) # was np.random.randn(1000)
```

Essential for generating test data, initializing embeddings, or creating synthetic training examples.

Understanding Array Shape and Reshape

Array shape defines the dimensions of your data structure—a fundamental concept when preparing batched inputs for LLM processing. The shape attribute returns a tuple indicating size along each dimension.

```
arr = np.arange(12)
print(arr.shape) # (12,)

reshaped = arr.reshape(3, 4)
print(reshaped.shape) # (3, 4)

# Automatic dimension calculation
auto = arr.reshape(2, -1)
print(auto.shape) # (2, 6)
```

The `reshape()` method reorganizes array elements without copying data—it returns a new view. Use `-1` to let NumPy calculate one dimension automatically, ensuring total elements remain constant.

Common pitfall: Reshaping fails if the total number of elements doesn't match. An array with 12 elements cannot reshape to (3, 5) because $3 \times 5 = 15$.

For LLM pipelines, reshaping is crucial when converting flat token sequences into batched matrices or splitting embedding vectors into attention head dimensions.

Vectorized Operations: The Performance Game-Changer



Element-wise Operations

```
arr = np.array([1, 2, 3, 4])  
result = arr * 2 + 10  
# [12, 14, 16, 18]
```

Operations apply to all elements simultaneously without explicit loops.



Mathematical Functions

```
arr = np.array([1, 4, 9, 16])  
sqrt = np.sqrt(arr)  
log = np.log(arr)  
exp = np.exp(arr)
```

Universal functions (ufuncs) operate element-wise with optimized C implementations.



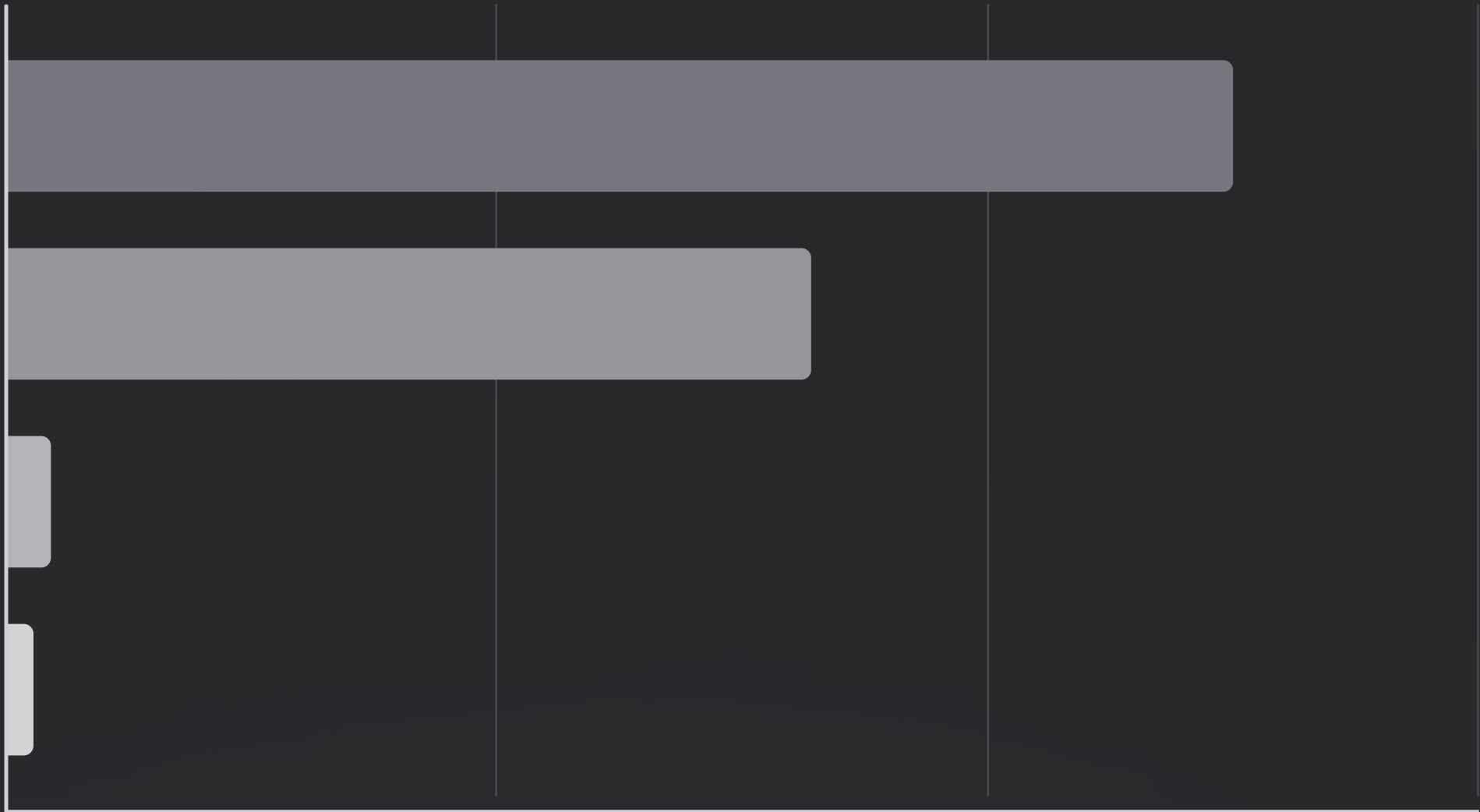
Boolean Operations

```
arr = np.array([1, 2, 3, 4, 5])  
mask = arr > 3 # Boolean array  
filtered = arr[mask] # [4, 5]
```

Create masks for filtering, validation, or conditional processing in pipelines.

Vectorized operations eliminate Python loop overhead and use optimized execution for dramatic speed improvements. When processing text embeddings or token arrays for LLMs, always think vectorized first.

Vectorization vs. Loops: Real Performance Impact



The Loop Problem

```
# Slow: Python loop
result = []
for x in data:
    result.append(x * 2 + 10)

# Much faster: Vectorized
result = data * 2 + 10
```

This benchmark shows processing 1 million floating-point numbers. The vectorized NumPy approach achieves nearly **45x speedup** over pure Python loops—the difference between waiting 20 seconds versus half a second.

For LLM preprocessing tasks like tokenization normalization or embedding transformations across millions of records, vectorization isn't optional—it's essential for practical performance.

Memory Footprint Considerations

24

Bytes per Python List Element

Python list objects carry significant overhead due to pointer storage and object metadata

8

Bytes per NumPy Float64

Contiguous memory layout with fixed data types enables compact storage

75%

Memory Reduction

Switching from lists to NumPy arrays can reduce memory usage by over two-thirds

Understanding memory footprint becomes critical when working with large-scale LLM datasets. A training corpus with 10 million token embeddings (768 dimensions each) consumes approximately 58 GB as float64 NumPy arrays versus 246 GB+ as Python lists.

❏ **Pro tip:** Use `arr.nbytes` to check memory consumption of NumPy arrays. Consider float32 instead of float64 for embeddings when precision allows—cutting memory usage in half with minimal accuracy impact for most LLM applications.

Jupyter Notebooks: Your Data Pipeline Workshop

Jupyter notebooks provide an interactive development environment that combines executable code, rich visualizations, and narrative documentation in a single interface. This integration makes them ideal for exploratory data analysis and iterative pipeline development.



Literate Programming

Weave markdown explanations with code cells to document your analytical reasoning and pipeline logic as you build.



Immediate Feedback

Execute cells individually to inspect intermediate results, validate transformations, and debug issues iteratively.



Shareable Analysis

Export complete notebooks as HTML, PDF, or share via GitHub to communicate findings with your team.



Essential Jupyter Magic Commands

%matplotlib inline

Renders plots directly below code cells instead of separate windows. Essential for creating reproducible visual analyses that display inline with your narrative.

```
%matplotlib inline
import matplotlib.pyplot as plt
plt.plot([1, 2, 3, 4])
```

%%time and %timeit

Measure execution time to identify bottlenecks. Use %%time for single runs or %timeit for statistical averages across multiple executions.

```
%%time
result = slow_function(large_dataset)
```

%load_ext and %autoreload

Load extensions and automatically reload modified modules without restarting the kernel—crucial when developing pipeline utilities in separate files.

```
%load_ext autoreload
%autoreload 2
```

%who and %whos

List variables in the current namespace. %whos provides detailed information including type, size, and memory usage—helpful for tracking large arrays.

```
%whos ndarray # Show only NumPy arrays
```


Building Reproducible Data Pipelines

Reproducibility Checklist

- **Set random seeds:** Use `np.random.default_rng(42)` at the notebook start to ensure consistent results across runs
- **Version dependencies:** Document library versions with `pip freeze` or include `requirements.txt`
- **Clear execution order:** Run "Restart & Run All" regularly to verify cells execute sequentially
- **Parameterize inputs:** Define configuration variables in early cells rather than hardcoding values throughout
- **Document assumptions:** Use markdown cells to explain data sources, transformation logic, and validation criteria



Reproducibility ensures that your data preprocessing pipeline produces identical outputs when re-executed—critical for LLM training where even small data variations can impact model performance.

Structure your notebooks with clear sections: imports and setup, data loading, exploratory analysis, transformation pipeline, validation, and export. This organization makes pipelines easier to maintain and transition to production scripts.

Markdown Narratives: Telling Your Data Story

Effective data analysis requires more than working code—it demands clear communication of insights, methodology, and rationale. Markdown cells in Jupyter transform notebooks from mere code containers into comprehensive analytical documents.

Structure Your Analysis

Use headers (# ## ###) to create logical sections. Start with problem statement, document each transformation step, and conclude with findings and next steps.

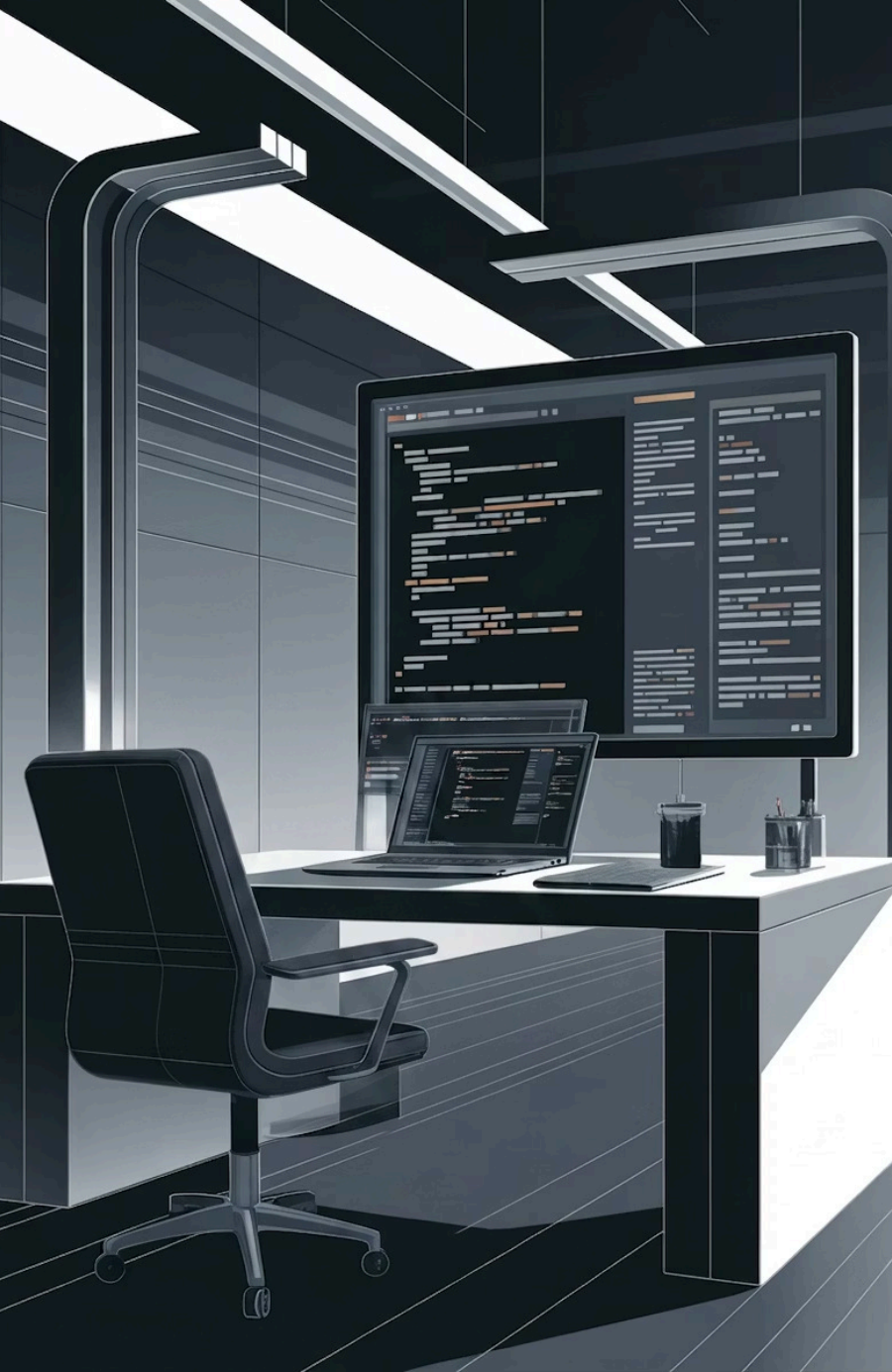
Explain Decisions

Document *why* you chose specific approaches: "We use StandardScaler instead of MinMaxScaler because our embeddings contain outliers that would distort the range."

Annotate Results

Don't just display outputs—interpret them. "This 15% null rate in the description field requires imputation strategy before fine-tuning."

📌 **Best practice:** Write markdown *before* implementing code. This "narrative-first" approach clarifies your analytical thinking and often reveals logical gaps early. Your future self (and teammates) will thank you for the documentation.



Lab 1 Overview: Hands-On Practice

First lab applies everything we've covered to a practical scenario: preprocessing a text dataset for LLM fine-tuning. You'll extract data from multiple sources, transform it using NumPy operations, and create a reproducible Jupyter workflow.



Extract

Load sample text data and metadata from CSV and JSON sources using pandas



Transform

Clean text, vectorize operations with NumPy, and validate data quality constraints



Load

Export preprocessed dataset in formats compatible with LLM training frameworks

Key Takeaways



NumPy is your performance foundation

Vectorized operations and efficient memory usage make NumPy essential for processing large-scale LLM datasets. Always think arrays over lists, vectorization over loops.



Reproducibility isn't optional

Set random seeds, document dependencies, parameterize configurations, and maintain clear execution order. These practices prevent debugging nightmares and ensure consistent pipeline behavior.



Jupyter enables iterative development

Interactive cells, inline visualizations, and markdown narratives create a powerful environment for exploratory analysis and pipeline prototyping before production deployment.



Document as you build

Write markdown explanations for your analytical decisions and transformation logic. Future you—and your teammates—will appreciate the context when revisiting pipelines weeks, months, or years later.



Thanks